

AI KNOWLEDGE GRAPH & SKILL INTELLIGENCE PLATFORM

Complete Project Documentation



AIKG
Knowledge Graph

TEAM MEMBERS

MALAIKA NAZ

Backend, Neo4j Knowledge Graph & Integration

SABA HAYAT

Frontend Development & UI Integration

Technologies: React + Vite • FastAPI • Python • Neo4j • Cypher • JWT • Axios • Tailwind CSS • Recharts • jsPDF

1. Problem Statement

The case study focused on an ecosystem in which students, mentors, skills, technologies, projects, products, certificates, case studies and learning resources existed as separate records. The main challenge was not simply storing these records, but understanding the relationships between them and turning those relationships into useful information for students, mentors and HR.

- Disconnected data made it difficult to understand a student's complete learning and project profile.
- Traditional record-based filtering could not naturally answer relationship questions across multiple entities.
- HR needed a single place to view students, mentors, skills, technologies, analytics and reports.
- Students needed meaningful recommendations rather than isolated lists of resources or projects.
- The system needed authentication and protected HR access.
- The final application had to be functional from the frontend to the backend and database, not only visually complete.

2. Solution Provided

We solved the problem by designing a graph-backed full-stack platform. Neo4j was used as the relationship layer, FastAPI/Python as the REST API and business-logic layer, and React as the interactive client. CSV files were converted into graph nodes and relationships. Reusable Cypher queries were then used for profiles, analytics, recommendations and HR reporting.

- Neo4j models entities and their relationships explicitly.
- FastAPI exposes verified graph queries through REST APIs.
- JWT authentication protects HR routes and role-based authorization restricts access.
- Recommendation functions connect student skills to projects, mentors, resources and similar students.
- React pages consume the APIs and provide searchable, navigable modules.
- HR workflows include dashboard KPIs, student/mentor management, analytics and reports.
- jsPDF is used for clean document/certificate generation instead of relying on screenshots.

3. Main Objectives

- Build a structured Knowledge Graph for the Ezitech ecosystem.
- Expose graph intelligence through REST APIs.
- Implement secure HR authentication and authorization.
- Provide recommendation and skill-intelligence features.
- Create functional dashboards and management modules.
- Generate professional reports/certificates.
- Validate the system through Neo4j queries, API testing and frontend flows.

4. Project Overview

The completed platform combines a knowledge graph, API layer, recommendation engine and web interface into one system. The final application supports both general platform users and an authenticated HR workflow.

Module	Purpose
Authentication	Signup, login, JWT token handling, logout and protected HR access.
Dashboard	Platform KPIs and overview with navigation to related modules.
Students	Student list, search, profile, skills, projects and certificates.
Mentors	Mentor list and mentor/student relationship views.
Skills	Explore skills represented in the graph.
Technologies	Explore technologies and their graph relationships.
Projects	Project list and project details.
Learning Resources	Learning resources connected to skills and recommendations.
Certificates	Certificate viewing and PDF generation workflow.
Products	Products connected to projects.
Case Studies	Case-study records and project relationships.
Recommendations	Recommended skills, projects, mentors, resources and similar students.
HR Dashboard	Authenticated HR overview and operational KPIs.
HR Students	Student management, search and profile access.
HR Mentors	Mentor management and related information.
HR Analytics	Graph-driven skill and technology analytics.
HR Reports	Structured HR summary and report generation.

4.1 Main User Flows

- General user: Dashboard → browse modules → open details → recommendations → resources/projects/certificates.
- HR: Login → HR Dashboard → KPI/module navigation → Students/Mentors → Student Profile → Analytics/Reports → Logout.
- Authentication: Signup → Login → JWT returned → token stored → protected requests receive Bearer token → Logout clears authentication state.

5. System Architecture

The final application follows a layered architecture. The React client handles presentation and interaction, FastAPI handles REST APIs and application logic, Neo4j stores the connected knowledge model, CSV files provide structured data, and JWT protects HR routes.

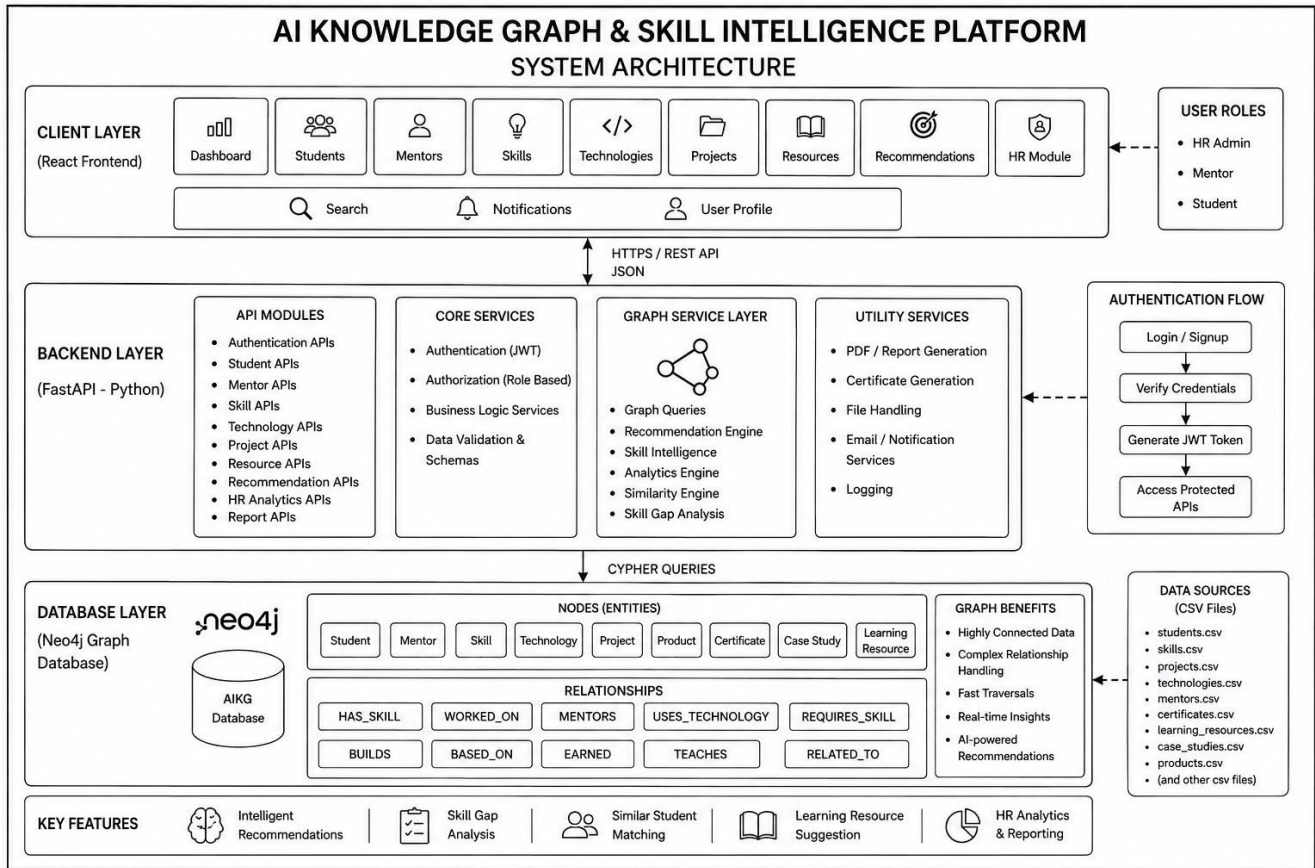


Figure 1. Final black-and-white system architecture of the platform.

Layer	Main Responsibilities
Client Layer	React + Vite pages, components, routing, Axios services, authentication context and UI actions.
Backend Layer	FastAPI routers, Pydantic schemas, authentication utilities, graph queries, recommendation functions and reports.
Database Layer	Neo4j graph database named aikg; nodes and relationships queried using Cypher.
Data Layer	Entity CSV files and relationship CSV files used to populate the graph.
Security Layer	Password hashing/verification, JWT creation/decoding, HTTP Bearer authentication and HR role authorization.

5.1 Technology Stack

Layer	Technology
Frontend	React, Vite, React Router, Tailwind CSS, Axios, React Icons/Lucide React, Recharts
Backend	Python, FastAPI, Uvicorn, Pydantic
Database	Neo4j and Cypher

Layer	Technology
Authentication	JWT, HTTP Bearer, password hashing/verification
Data	CSV datasets and relationship CSV files
Reporting	jsPDF and browser-side document generation; html2canvas where required
Development	VS Code, PowerShell and Neo4j Desktop

6. Knowledge Graph Data Model

The graph represents the domain as entities plus explicit relationships. This allows multi-hop questions and provides the foundation for analytics and recommendations.

6.1 Main Node Types

- Student
- Mentor
- Skill
- Technology
- Project
- Product
- Certificate
- CaseStudy
- LearningResource

6.2 Main Relationships

Relationship	Meaning
HAS_SKILL	Student → Skill
WORKED_ON	Student → Project
MENTORS	Mentor → Student
USES_TECHNOLOGY	Project → Technology
REQUIRES_SKILL	Project → Skill
BUILDS	Project → Product
BASED_ON	Project → CaseStudy
EARNED	Student → Certificate
TEACHES	LearningResource → Skill

6.3 Dataset Scale

The populated graph contains approximately 50 Student nodes, 10 Mentor nodes, 20 Skill nodes, 15 Technology nodes, 20 Project nodes, 5 Product nodes, 5 Certificate nodes, 5 CaseStudy nodes and 5 LearningResource nodes. Relationship CSV files connect these entities into the working graph.

6.4 CSV Data Files

- Entity files: students.csv, mentors.csv, skills.csv, technologies.csv, projects.csv, products.csv, certificates.csv, case_studies.csv, learning_resources.csv.
- Relationship files: student_skills.csv, student_projects.csv, mentor_students.csv, project_technologies.csv, technology_skills.csv, project_products.csv, project_case_studies.csv, student_certificates.csv, learning_resource_skills.csv.

7. Backend Implementation

The backend separates API routing, request validation, authentication, graph access and recommendation logic. This separation made it possible to debug individual layers without rewriting the complete application.

- FastAPI REST APIs for platform and HR functionality.
- Pydantic schemas for signup/login request validation.
- Password hashing and verification.
- JWT token creation and decoding.
- HTTP Bearer authentication for protected endpoints.
- Role-based HR authorization through `require_hr`.
- Reusable GraphQL queries and recommendation functions.
- JSON responses for dashboards, lists, profiles, analytics, recommendations and reports.

7.1 Authentication APIs

Method	Endpoint	Purpose
POST	/auth/signup	Create an HR account with the HR role.
POST	/auth/login	Validate credentials and return JWT access token plus user information.

7.2 HR APIs

Method	Endpoint	Purpose
GET	/hr/profile	Verify authenticated HR access and return user information.
GET	/hr/dashboard	Return HR dashboard statistics.
GET	/hr/students	Return all students.
GET	/hr/students/{student_id}	Return a student's profile, skills, projects and certificates.
GET	/hr/mentors	Return all mentors.
GET	/hr/mentors/{mentor_id}	Return mentor/student relationship information.
GET	/hr/analytics	Return top skills and technologies.
GET	/hr/recommendations/{student_id}	Return student recommendation data.
GET	/hr/reports	Return HR summary, skills, technologies and recent records.

7.3 Important Backend Files

- `backend/app/api/v1` — API routers.
- `backend/app/auth` — JWT and password utilities.
- `backend/app/schemas` — request validation schemas.
- `backend/app/graph` — Neo4j driver, graph schema, loader, graph queries and recommendations.

- `backend/app/main.py` — FastAPI application entry point.

8. Frontend Implementation

The frontend was implemented with React + Vite and organized into pages, reusable components, API service files, authentication context and protected routing.

- React Router handles public, general and HR routes.
- AuthContext stores login state and token information.
- ProtectedRoute prevents unauthorized HR access.
- Axios services connect each page to FastAPI.
- Search, navigation, KPI cards, profile actions, reports and certificate actions were made functional.
- HR pages use a dedicated HR folder with consistent imports and routes.
- The final UI was tested through real API responses rather than only static dummy content.

8.1 Main Frontend Pages

Area	Pages
Authentication	Login, Signup
Main Platform	Dashboard, Students, Mentors, Skills, Technologies, Projects, ProjectDetails, LearningResources, Certificates, Products, CaseStudies, Recommendations
HR	HRDashboard, HRStudents, HRStudentProfile, HRMentors, HRAnalytics, HRReports

8.2 Functional Navigation

- Dashboard KPI cards navigate to their respective modules.
- HR Students opens student profiles from the student cards.
- Student profile exposes graph-backed skills, projects and certificate generation.
- HR Analytics loads API-backed graph analytics.
- HR Reports generates a structured report rather than a full-screen screenshot.
- Logout clears the token and returns to /login.
- After logout, manually opening /hr/dashboard redirects to login because the route is protected.

9. Recommendation & Skill Intelligence

The recommendation layer uses graph relationships to produce useful suggestions. Instead of matching only on isolated fields, the system traverses connected student, skill, project, resource and mentor relationships.

- Recommended skills based on the student's existing graph profile.
- Recommended projects connected to relevant skills/technologies.
- Recommended learning resources connected to skills.
- Similar students based on overlapping skills/relationships.
- Recommended mentors using graph-backed relevance.
- Skill-gap analysis for HR/student intelligence.

10. HR Intelligence Module

The HR module provides a protected operational layer over the same knowledge graph. HR can view dashboard KPIs, students, mentors, analytics and reports after successful authentication.

- HR Dashboard — overview and KPI navigation.
- HR Students — searchable student list and profile access.
- HR Student Profile — student information plus graph-backed skills/projects/certificates.
- HR Mentors — mentor list and relationships.
- HR Analytics — top skills and technologies.
- HR Reports — structured summaries and recent records.
- JWT + role checking ensures that these routes are not publicly accessible.

11. Reporting & Certificate Generation

A major improvement was replacing simple browser screenshots with document generation. HR reports are formatted as clean documents, while certificates are generated as PDFs with the student's name clearly displayed, an appreciation statement and HR signature space.

- Project/report content is presented as structured text and tables.
- Student certificates are generated in PDF form using jsPDF.
- The certificate design places the student's name prominently in the center.
- A short appreciation statement is included.
- HR signature space is placed clearly at the bottom.
- Project information can be opened separately from the certificate workflow.

12. Important Neo4j / Cypher Queries

These queries are suitable for demonstrating the Knowledge Graph to a supervisor. They show the graph itself, entity counts, relationships and analytics.

12.1 Overall Graph

```
MATCH (n)-[r]->(m)
RETURN n, r, m
LIMIT 100;
```

12.2 Node Type Counts

```
MATCH (n)
RETURN labels(n) AS NodeType, count(n) AS Count
ORDER BY Count DESC;
```

12.3 Student → Skill

```
MATCH (s:Student)-[r:HAS_SKILL]->(sk:Skill)
RETURN s, r, sk
LIMIT 100;
```

12.4 Student → Project

```
MATCH (s:Student)-[r:WORKED_ON]->(p:Project)
RETURN s, r, p
LIMIT 100;
```

12.5 Mentor → Student

```
MATCH (m:Mentor)-[r:MENTORS]->(s:Student)
RETURN m, r, s
LIMIT 100;
```

12.6 Top Skills

```
MATCH (s:Student)-[:HAS_SKILL]->(sk:Skill)
RETURN sk.name AS Skill, count(s) AS Students
ORDER BY Students DESC
LIMIT 10;
```

12.7 One Student's Skills

```
MATCH (s:Student {student_id: "STU001"})-[:HAS_SKILL]->(sk:Skill)
RETURN s, sk;
```

12.8 Skill → Project Multi-Hop

```
MATCH (s:Student {student_id: "STU001"})-[:HAS_SKILL]->(sk:Skill)-[:REQUIRES_SKILL]-(p:Project)
RETURN s, sk, p
LIMIT 50;
```

13. Graph Evidence & Query Outputs

The following figures were taken from the project's Neo4j exploration and are included as visual evidence of nodes, relationships and student-centered traversals.

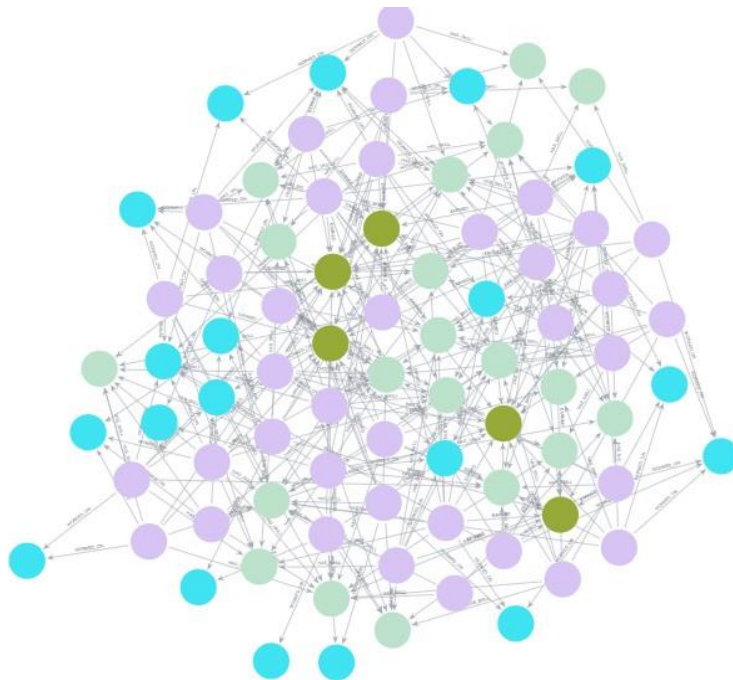


Figure 2. Overall Knowledge Graph view.

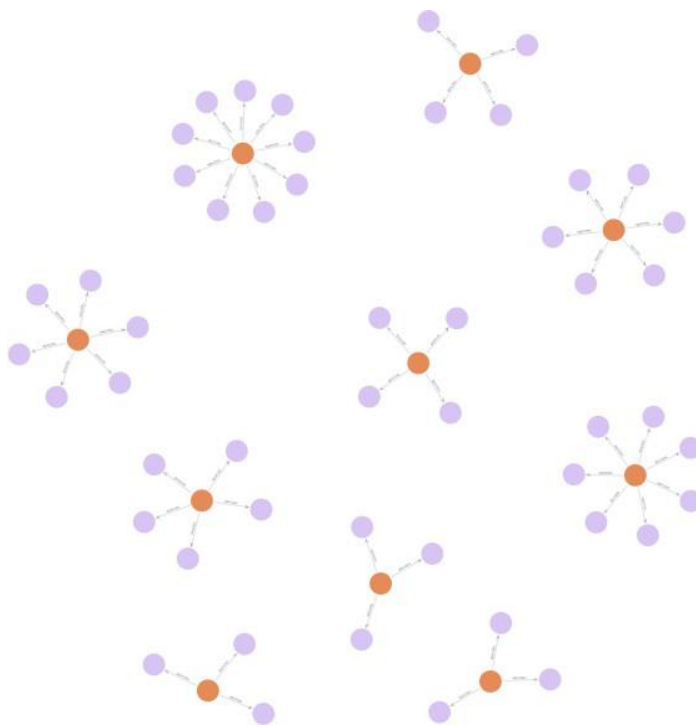


Figure 3. Mentor-student relationship view.

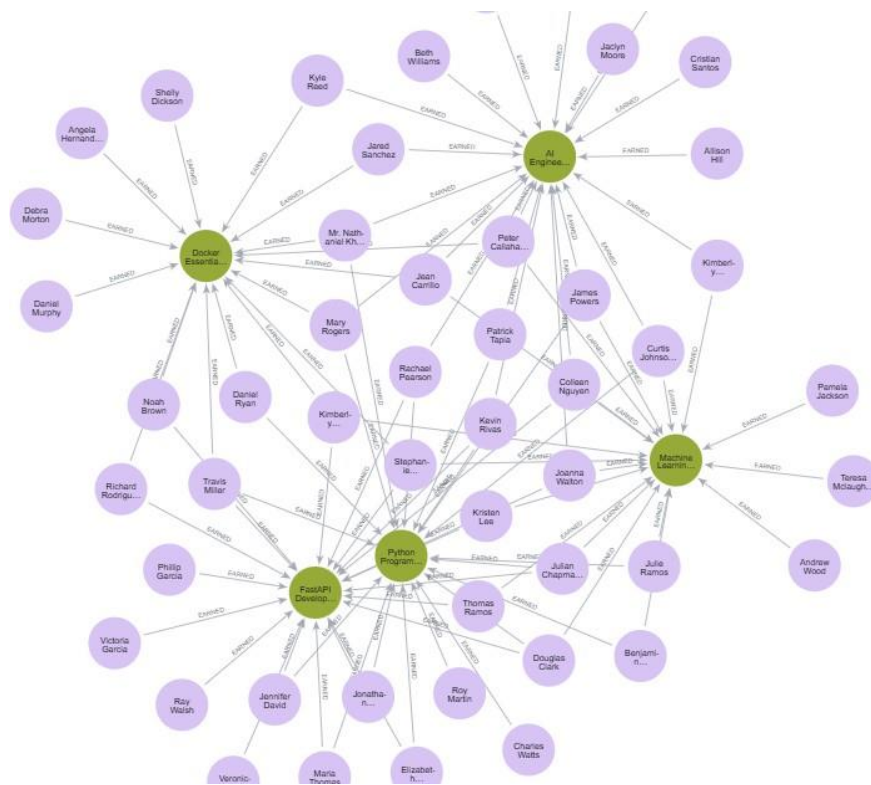


Figure 4. Project/case-study relationship exploration.

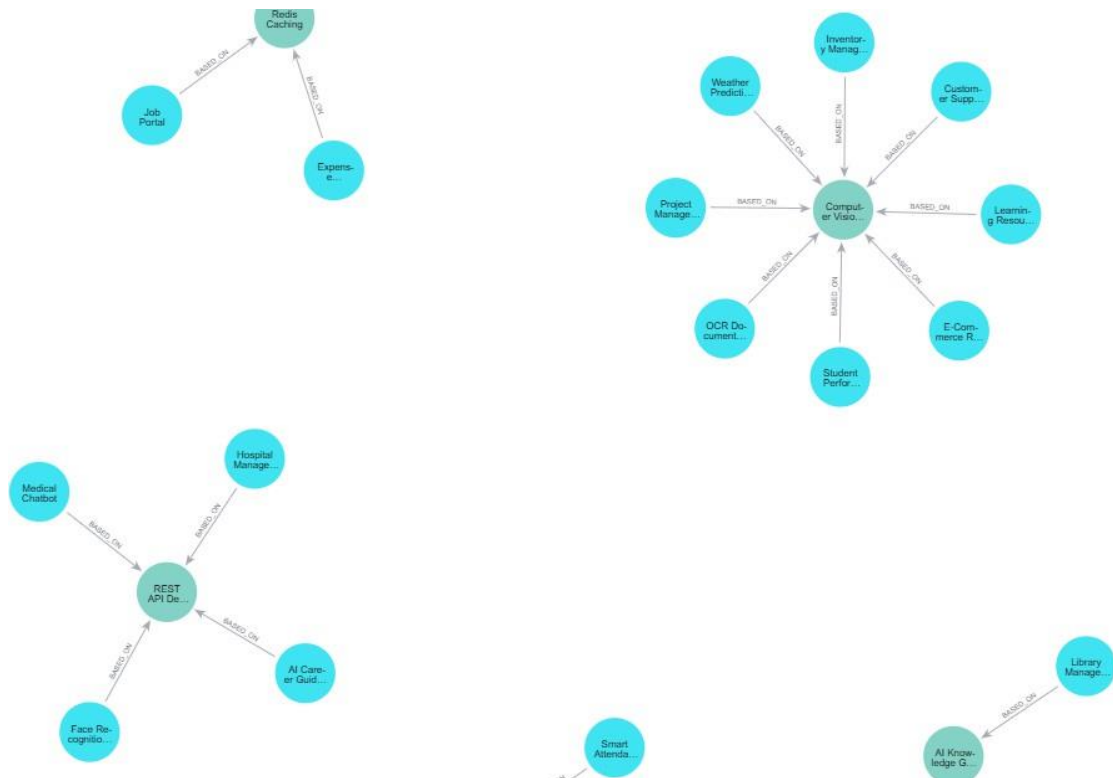


Figure 5. Student–certificate relationship exploration.

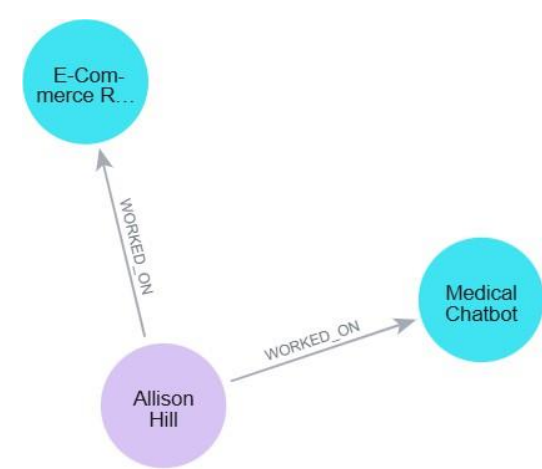


Figure 6. Student–project relationship exploration.

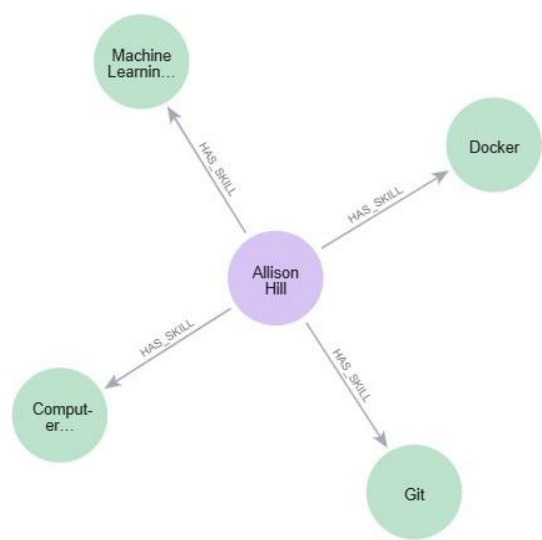


Figure 7. Student–skill relationship exploration.

Problems Faced During Development & Their Solutions

During development, the main difficulty was integrating several technologies at the same time: FastAPI, Neo4j, JWT authentication, React routing, Axios services and UI components. The table below records the practical issues that occurred and how they were resolved.

13.1 Backend Issues

Issue	What Happened / Impact	How We Solved It	Status
Module / import errors	Backend packages were not resolved consistently, preventing clean startup.	Standardized the package structure and used the correct Uvicorn entry point: <code>backend.app.main:app</code> .	Resolved
Neo4j driver/database mismatch	Graph query code expected a database attribute that was not consistently available.	Centralized Neo4j driver/database configuration and aligned all graph-query classes with the same database.	Resolved
Neo4j result consumption error	Query results were consumed incorrectly, causing runtime failures.	Corrected result handling and converted Neo4j records into the Python structures expected by the APIs.	Resolved
Wrong graph property keys	Some Cypher queries returned empty data because property names did not match the loaded CSV schema.	Checked the actual node properties and aligned Cypher queries with the real Neo4j data model.	Resolved
Recommendation queries returned empty lists	Some recommendation cards had no results for the selected student.	Checked the underlying relationships and adjusted traversals/recommendation queries to use the available graph connections.	Resolved
JWT authentication problems	Protected APIs required a consistent token flow and correct Bearer authentication.	Implemented token creation/decoding, HTTP Bearer verification and <code>require_hr</code> role checking.	Resolved
API response-shape mismatch	Frontend expected arrays/objects that did not match the actual JSON returned by FastAPI.	Standardized frontend service/page handling around the real API response structure.	Resolved

13.2 Frontend Issues

Issue	What Happened / Impact	How We Solved It	Status
Login returned HTTP 422	FastAPI rejected the login body because the frontend payload did not match <code>LoginRequest</code> .	Aligned the Axios login payload exactly with <code>email + password</code> .	Resolved
React rendered a validation object	FastAPI validation details were objects such as <code>type/loc/msg/input</code> and React could not render them directly.	Converted error details into safe user-readable strings before rendering.	Resolved
Empty white page after login	The login request could fail or the destination route could crash during rendering.	Checked browser console errors, <code>AuthContext</code> , route navigation and protected-route behavior; corrected the failing integration.	Resolved
HR pages did not open	Routes, imports and HR service paths were not completely aligned.	Unified <code>App.jsx</code> HR routes, <code>ProtectedRoute</code> usage, HR service endpoints and the capitalized HR folder structure.	Resolved
HR Analytics 'not found'	Frontend analytics request did not match the backend route.	Aligned <code>getHRAnalytics()</code> with <code>GET /hr/analytics</code> and removed the conflicting service definition.	Resolved
Duplicate <code>getHRAnalytics</code> export	The same function was declared twice in <code>hrApi.js</code> , causing module-level conflicts.	Removed the duplicate and retained one canonical <code>getHRAnalytics</code> function.	Resolved

Issue	What Happened / Impact	How We Solved It	Status
Lucide GitHub import error	The installed lucide-react package did not expose the requested Github export in the current build.	Removed the incompatible named import and adjusted the GitHub action to use a supported icon/action implementation.	Resolved
GitHub button opened 404	A placeholder/dummy external link did not point to a verified public repository/profile.	Kept the UI functional and documented the link as a placeholder to be replaced with the verified URL before public deployment.	Known
Report printed as full-screen screenshot	Browser print/capture produced an image-like page rather than a clean document.	Introduced structured PDF generation with jsPDF so reports are generated as documents instead of screenshots.	Resolved
Certificate was only a dummy button	The profile initially showed a button without a proper certificate document.	Implemented PDF certificate generation with prominent student name, appreciation text and HR signature space.	Resolved

14. Testing & Validation

Testing was performed at graph, backend API and frontend levels. Final validation focused on both functionality and presentation.

14.1 Backend Validation

- FastAPI starts successfully with Uvicorn.
- Swagger/OpenAPI loads at /docs.
- Signup creates an HR account.
- Login returns a JWT access token and user object.
- Protected HR routes reject missing/invalid authentication.
- HR role authorization is enforced.
- Students, mentors, analytics and reports return graph-backed data.
- Neo4j queries return expected nodes and relationships.

14.2 Frontend Validation

- Login and signup pages render correctly.
- Successful login reaches the correct dashboard.
- Sidebar navigation opens each module.
- HR dashboard KPI cards navigate to related modules.
- Student search and profile navigation work.
- Analytics and reports load API data.
- Search and notification controls respond.
- Logout clears the session and returns to /login.
- Protected HR URLs redirect to login after logout.
- Certificate/report generation produces document output.
- Production build completes successfully.

14.3 Production Build Result

The final Vite build completed successfully with 2720 modules transformed. The build generated the expected HTML, CSS and JavaScript assets. Vite reported a warning that one minified JavaScript chunk was larger than 500 kB. This is a performance warning, not a build failure.

15. How to Run the Project

15.1 Backend

- Open the project root.
- Activate the Python virtual environment.
- Ensure the .env file contains the Neo4j configuration.
- Start the backend using the command below.

```
uvicorn backend.app.main:app --reload
```

Swagger/OpenAPI:

```
http://127.0.0.1:8000/docs
```

15.2 Frontend

- Open the frontend directory.
- Install dependencies.
- Start the development server.

```
npm install  
npm run dev
```

15.3 Production Build

```
npm run build
```

15.4 Important Configuration

- Keep Neo4j credentials and secrets in environment configuration.
- Use the same backend base URL in Axios services during local development.
- Ensure Neo4j is running before testing graph-backed pages.
- Replace placeholder external links with verified URLs before public deployment.

16. Final Deliverables

- Working React + Vite frontend.
- Working FastAPI backend.
- Neo4j Knowledge Graph with entity and relationship data.
- Graph query and recommendation layer.
- JWT authentication and protected HR module.
- HR dashboard, students, mentors, analytics and reports.
- Dashboard, Students, Mentors, Skills, Technologies, Projects, Learning Resources, Certificates, Products, Case Studies and Recommendations modules.
- Interactive student profiles with graph-linked information.
- PDF/report and certificate generation workflow.
- Swagger/OpenAPI backend documentation.
- Successful production frontend build.

17. Final Outcome

The completed system converts the original disconnected-data problem into an integrated graph-backed web platform. Neo4j provides the relationship layer, FastAPI exposes that intelligence through APIs, and React turns the APIs into a usable application for general users and HR staff.

18. Conclusion

The project was completed as a full-stack AI and Knowledge Graph application rather than a collection of isolated screens. It combines graph modeling, Cypher, FastAPI APIs, authentication, recommendations, analytics, reporting and React interaction into one coherent platform. The development process also addressed real backend and frontend integration problems, and the final system was validated through API testing, Neo4j queries, navigation flows and a successful production build.

End of Documentation